Projet Semestriel

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	✓
CO2	Understand process scheduling, memory management, and file system organization algorithms	✓
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	✓

Introduction

Le projet semestriel est un travail **intégrateur** qui mobilise les trois résultats d'apprentissage du module (CO1, CO2, CO3) : les fondamentaux et la ligne de commande Linux, les sous-systèmes du noyau (processus, mémoire, fichiers) et l'administration/automatisation. Il vous demande de concevoir et de développer, en groupe, un outil Linux complet que vous **défendrez oralement** en fin de semestre.

Quatre projets vous sont proposés, de difficultés variées : le tableau de bord système (`sysdash`) couvre l'observation et l'administration du système ; les mini-shells (`opsh` en Python, `ocsh` en C) vous font reconstruire un interpréteur de commandes ; enfin, le noyau simulé (`esinos`) reproduit l'ordonnancement, la mémoire et le système de fichiers d'un SE.

Chaque projet vous amènera à découvrir de **nouveaux concepts** et de **nouvelles commandes** qui ne figurent pas au programme : c'est une **excellente occasion d'apprendre** par vous-même.

Difficulté et note maximale

Les projets varient en complexité : la **note maximale** atteignable dépend du projet choisi. Une soumission **parfaite** d'un projet plus difficile dépasse toujours une soumission parfaite d'un projet plus facile.

Projet	Sujet	Difficulté	Note max
Projet 1	`sysdash` — Tableau de bord système	**Intermediate**	17/20
Projet 2	`opsh` — Mini-shell (en Python)	**Challenging**	18/20
Projet 3	`ocsh` — Mini-shell (en C)	**Challenging**	19/20
Projet 4	`esinos` — Mini-noyau simulé	**Advanced**	20/20

Règles de constitution des groupes

* Groupes de **3 étudiants minimum 4 étudiants maximum**.
* **Les membres d'un groupe doivent appartenir au même groupe de TP.** Le mélange des sections n'est pas autorisé.
* Choix du projet via un **formulaire Microsoft** disponible dans le site du cours.

Format de la soutenance

* La soutenance a lieu pendant votre **séance de TP habituelle**.
* La discussion dure **15 minutes maximum** par groupe (démo, revue de code et questions).
* 4 cartes tirées au hasard : **2 CODE** et **2 CONCEPT**.
* Les 2 étudiants « CODE » expliquent le code ; les 2 « CONCEPT » expliquent les concepts.
* L'attribution est aléatoire : chaque membre doit maîtriser les deux volets.
* **Absence le jour de la soutenance = 0**, sans rattrapage.

Livrables

Soumettez un dépôt **GitHub public** contenant :

* `code/` — code source et scripts bien commentés
* `report.pdf` — 2 à 4 pages : approche, difficultés, résultats et liens avec le cours
* `README.md` — description de votre projet, accompagnée d'instructions d'exécution détaillées
* `demo.mp4` — une vidéo de démonstration, **caméra ouverte**, présentant le travail et expliquant le code

Projet 1 — `sysdash` : Tableau de bord système

Difficulté : Intermediate — Note max : 17/20

Objectif

Construire un tableau de bord Bash en **boucle** avec plusieurs options (monitoring, analyse de logs, audit), qui interroge le **système réel**.

Jalons (milestones)

1. Squelette : menu en boucle + sélection + gestion des arguments (`--help`)
2. Moniteur système : lecture `/proc` (CPU, mémoire, charge, processus), rafraîchissement, tri
3. Analyse de logs (`/var/log`) : top IP, 404, échecs d'authentification (`grep` / `sed` / `awk`)
4. Audit permissions : `/etc/passwd`, permissions, SUID/SGID, `umask`
5. Robustesse : gestion d'erreurs, codes de retour, sortie propre (`trap`)

Démo en soutenance

1. Menu → moniteur : CPU / mémoire / charge en direct, tri, sortie propre.
2. Logs (top IP, 404, échecs) ; audit (permissions faibles + explication du calcul octal).

Concepts de base (à couvrir)

* `/proc` ; états des processus ; load average ; RAM vs swap
* pipes/redirections comme IPC ; regex (BRE vs ERE) ; UID/GID ; notation octale ; `umask` ; SUID/SGID

Livrables

`sysdash.sh` (commenté) · `README.md` · `report.pdf` · `demo.mp4` (vidéo caméra ouverte)

Projet 2 — `opsh` : Mini-shell en Python

Difficulté : Challenging — Note max : 18/20

Objectif

Construire un interpréteur de commandes fonctionnel en Python : `opsh` lit une commande, la lance et supporte les mécanismes réels de Bash. Vous implémentez vos propres **commandes renommées** ainsi que les redirections, les pipes et les signaux.

Builtins		Alias	
commande bash	**commande opsh**	**commande bash**	**commande opsh**
`cd`	`go`	`ls`	`list`
`exit`	`quit`	`cat`	`show`
`pwd`	`path`	`cp`	`copy`
`echo`	`say`	`mv`	`move`
`help`	`info`	`rm`	`del`
`history`	`past`	`mkdir`	`newdir`
`pipe ( | )`	`pipe ( -> )`	`touch`	`newfile`

Le shell doit aussi fournir le builtin `osinfo`, qui affiche `Systèmes d'Exploitation -- UIR/ESIN`, et l'opérateur de pipe `->` (au lieu de `|`).

Jalons (milestones)

1. Boucle REPL : prompt `opsh>` → lecture → découpage par `->` → exécution via `os.fork()` / `subprocess`
2. Builtins et alias de la table ; redirections `>` `»` `<`
3. Pipes `->` ; arrière-plan `&` et signaux `SIGINT` / `SIGCHLD`
4. Variables et expansion : `$VAR`, `${VAR}`, `export`, `$?`, `$!`
5. Substitution `$(...)` / backticks ; `.opshrc` ; historique et complétion
6. Mode script `opsh script.opsh` (`if` / `for` / `while`) ; **optionnel** : job control `jobs` `fg` `bg`

Démo en soutenance

1. `opsh> list` · `go ..` puis `path` · `say hello > f.txt` puis `show f.txt`
2. `opsh> show f.txt -> wc -l` (pipe `->`) · `sleep 10 &` puis `ps` · `osinfo`

Concepts de base (à couvrir)

* Programme vs processus ; fork/exec/wait ; zombie / orphelin ; redirection (descripteurs 0/1/2) ; pipes comme IPC ; signaux
* Pourquoi `go` / `cd` doit être un builtin (non forkable) ; mode utilisateur vs noyau ; PATH / environnement

Livrables

`opsh` (code Python commenté) · `README.md` · `report.pdf` · `demo.mp4` (vidéo caméra ouverte)

Projet 3 — `ocsh` : Mini-shell en C

Difficulté : Challenging — Note max : 19/20

Objectif

Construire un interpréteur de commandes en C, en appelant directement les **appels système** du noyau : création de processus, redirections, pipes et signaux. Vous implémentez vos propres **commandes renommées** ainsi que le cœur de Bash.

Builtins		Alias	
commande bash	**commande ocsh**	**commande bash**	**commande ocsh**
`cd`	`go`	`ls`	`list`
`exit`	`quit`	`cat`	`show`
`pwd`	`path`	`cp`	`copy`
`echo`	`say`	`mv`	`move`
`help`	`info`	`rm`	`del`
`history`	`past`	`mkdir`	`newdir`
`pipe ( | )`	`pipe ( -> )`	`touch`	`newfile`

Le shell doit aussi fournir le builtin `osinfo`, qui affiche `Systèmes d'Exploitation -- UIR/ESIN`, et l'opérateur de pipe `->` (au lieu de `|`).

Jalons (milestones)

1. Boucle REPL : lecture → découpage par `->` → `fork()` → `execvp()` → `wait()`
2. Builtins et alias de la table ; redirections `>` `»` `<` via `dup2`
3. Pipes `->` (câblage avec `pipe`) ; arrière-plan `&` et signaux `SIGINT` / `SIGCHLD`
4. Variables et expansion : `$VAR`, `${VAR}`, `export`, `$?`, `$!`
5. Substitution `$(...)` / backticks ; `.ocshrc` ; historique et complétion
6. Mode script `ocsh script.ocsh` (`if` / `for` / `while`) ; **optionnel** : job control `jobs` `fg` `bg`

Démo en soutenance

1. `ocsh> list` · `go ..` puis `path` · `say hello > f.txt` puis `show f.txt`
2. `ocsh> show f.txt -> wc -l` (pipe `->`) · `sleep 10 &` puis `ps` · `osinfo`

Concepts de base (à couvrir)

* Programme vs processus ; fork/exec/wait ; pourquoi `exec` remplace l'image ; zombie / orphelin
* Pourquoi `go` / `cd` doit être un builtin (non forkable) ; pipe comme IPC ; signaux ; frontière appels système (user / kernel)

Livrables

`ocsh` (code C commenté + `Makefile`) · `README.md` · `report.pdf` · `demo.mp4` (vidéo caméra ouverte)

Projet 4 — `esinos` : Noyau de système d'exploitation simulé

Difficulté : Advanced — Note max : 20/20

Objectif

Une simulation en espace utilisateur d'un système d'exploitation avec un **ordonnanceur** et un **gestionnaire mémoire**, et en bonus un **mini système de fichiers**.

Jalons (milestones)

1. Ordonnanceur : FCFS / SJF / RR / Priority, diagramme de Gantt, temps d'attente/séjour moyens
2. Gestionnaire mémoire : pagination, traduction logique → physique, FIFO / LRU / Optimal, TLB et EAT
3. **Optionnel (bonus)** : mini système de fichiers (allocation à base d'inodes sur un fichier-disque)
4. Menu intégré + rapport

Démo en soutenance

1. Ordonnanceur : afficher les Gantt FCFS / SJF / RR.
2. Mémoire : montrer l'éviction FIFO pas à pas, puis comparer avec LRU.
3. **Bonus** : système de fichiers (créer / lire / supprimer, table d'inodes).

Concepts de base (à couvrir)

* FCFS / SJF / RR / Priority ; diagramme de Gantt ; temps d'attente / de séjour ; préemption
* Page vs cadre ; défaut de page ; FIFO / LRU / Optimal ; TLB et EAT ; mémoire virtuelle
* **Bonus** : inode et allocation de blocs

Livrables

code source commenté (C ou Python) · `README.md` · `report.pdf` · `demo.mp4` (vidéo caméra ouverte)